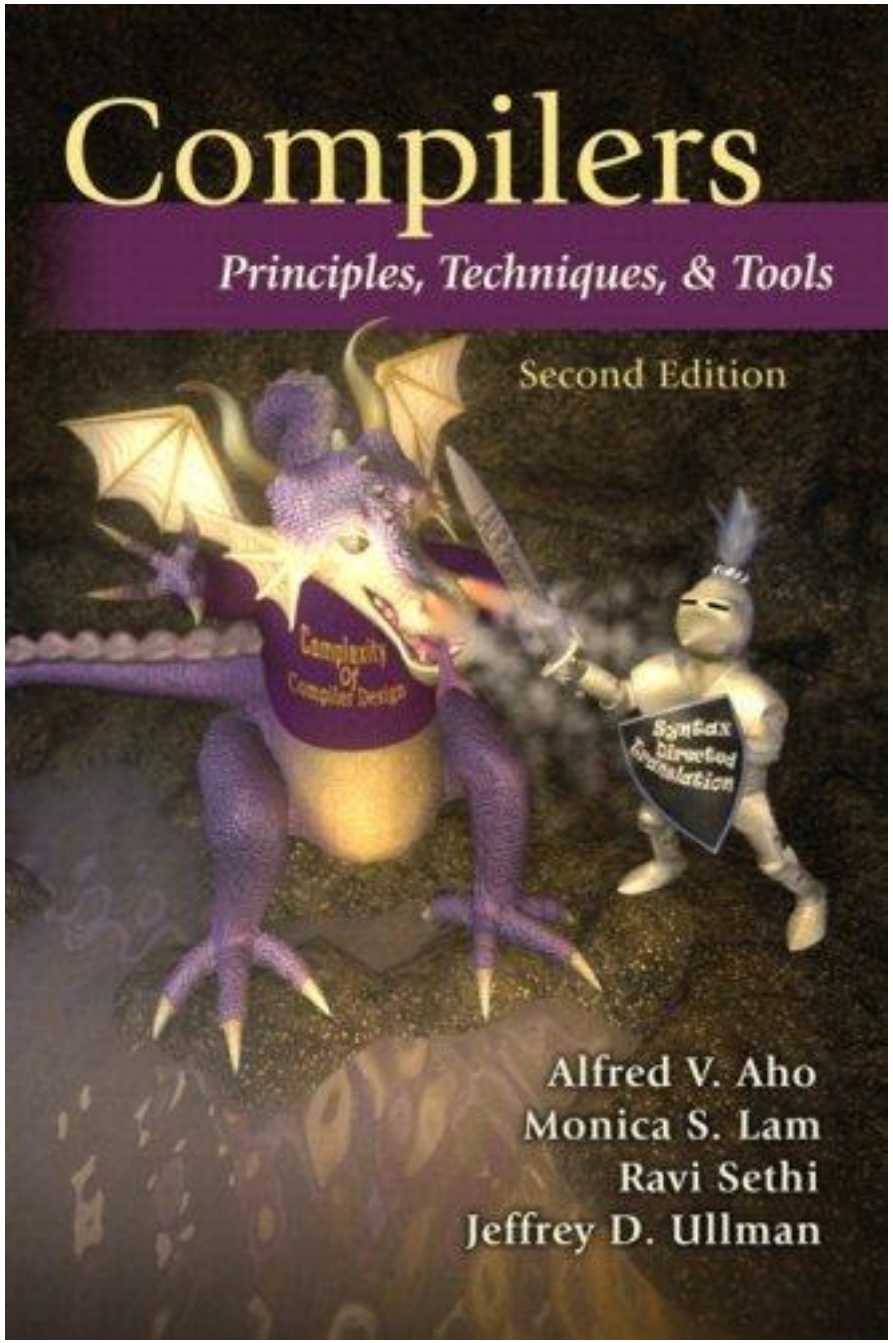
四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 6

Intermediate-Code Generation

Chapter 6

- intermediate representations
- static semantic checking
- intermediate code generation

CONTENTS

Static Checker

- 静态语义检查
  - 类型检查
  - 控制流检查
  - 唯一性检查
  - 作用域分析

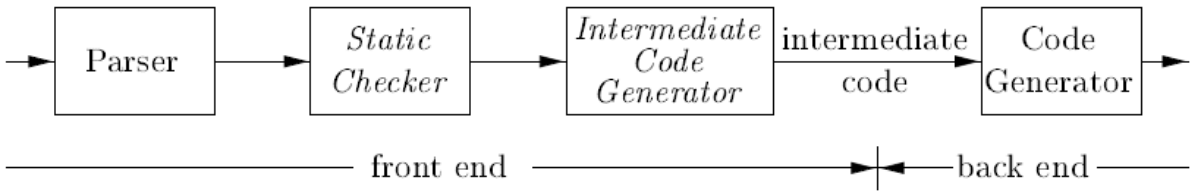


Figure 6.1: Logical structure of a compiler front end

```c
int gcd(int u,int v)
{
   do
   {
      int temp=v; v=u-u/v*v; u=temp;
   } while (v!=0);
   break;
   return;
}
```

```c
char str[100];
main() {
   int y,y=gcd(x);  //scanf("%d  %d",&x,&y);
   switch (y) {
       case 1: strcpy(str,"Hello"); x+=1;  break;
       case 1: str="Hello"+" World!"; break;
       default: x+=2; continue;
   }
}
```

Intermediate Representations

- 编译器在编译过程中可能使用一系列的中间表示形式。
  - 树型结构，例如抽象语法树
  - 线性表示形式，例如三地址码
  - 其他形式，例如逆波兰式、C语言等
  - 高层表示接近源语言，低层表示接近目标代码。

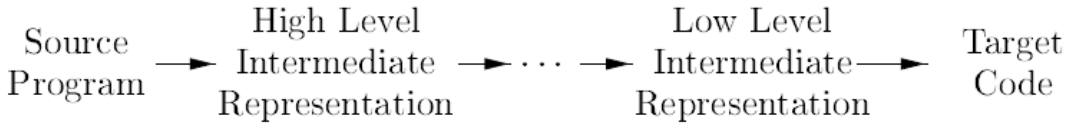
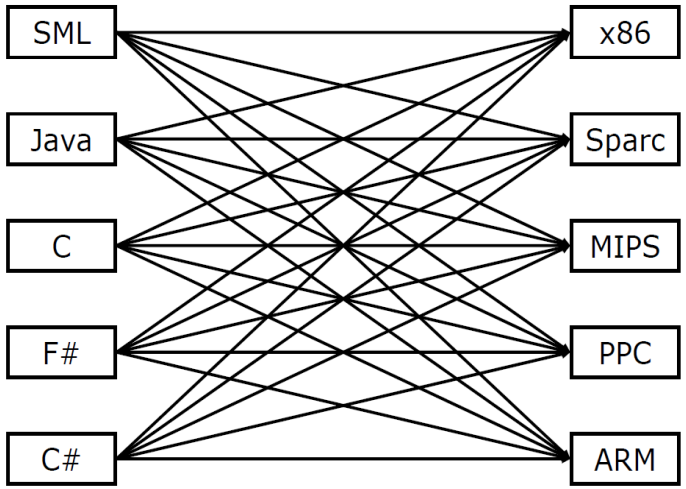
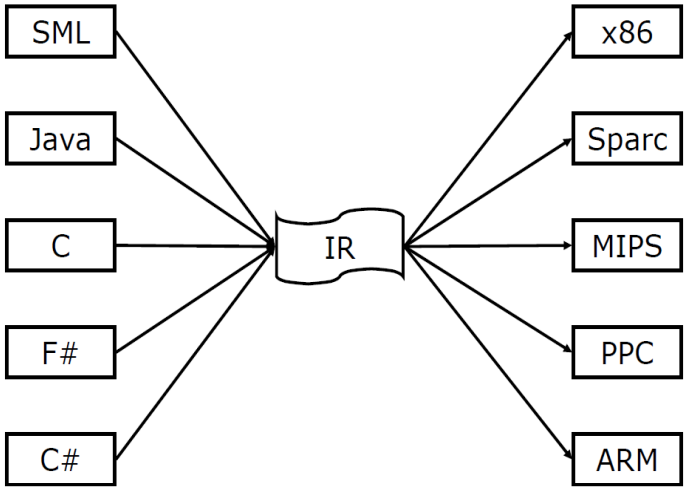


Figure 6.2: A compiler might use a sequence of intermediate representations





- 只要写出m种前端和n种后端，就可以得到m×n种编译器。

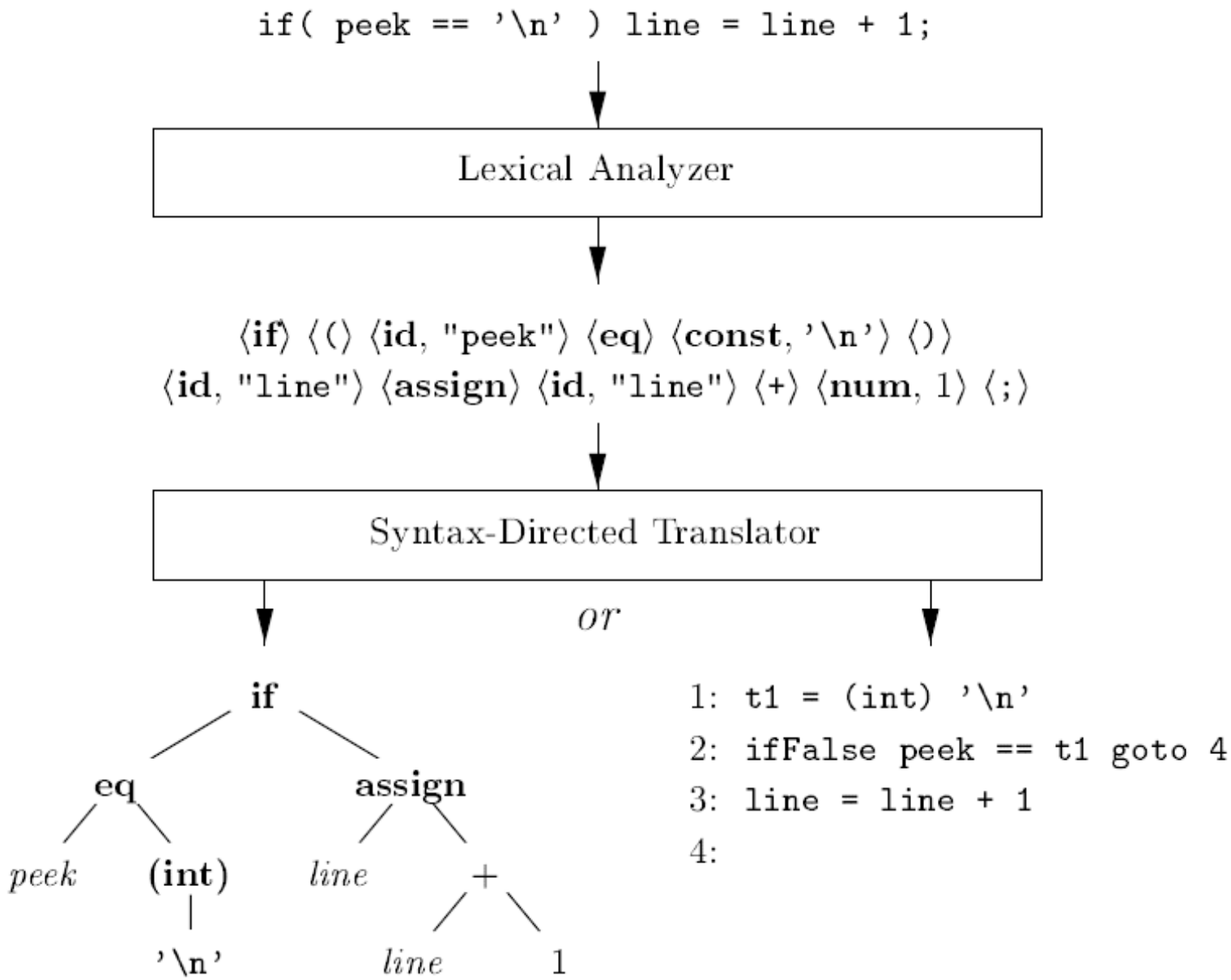
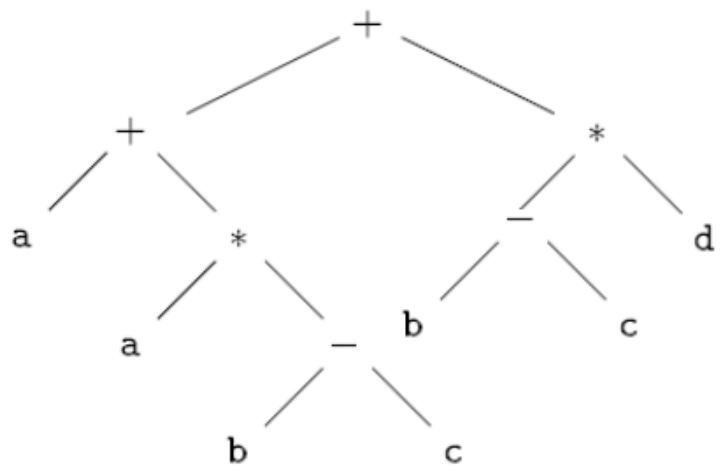


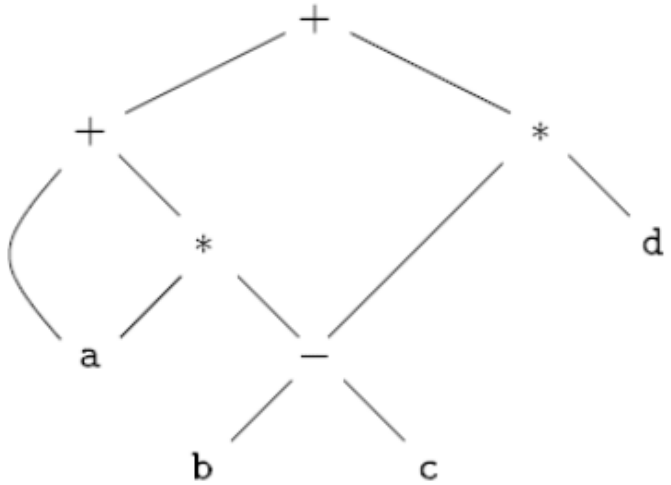
Figure 2.46: Two possible translations of a statement

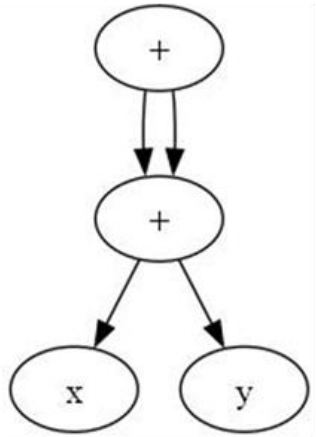
6.1 Variants of Syntax Trees

6.1.1 Directed Acyclic Graphs for Expressions

- 有向无环图DAG适用于处理表达式中的公共表达式。
- 在一个DAG中，代表公共子表达式的结点具有多个父结点。
- **Example 6.1** `a+a*(b-c)+(b-c)*d`







- **Exercise 6.1.2**

a) a+b+(a+b)　　b) a + b + a + b　　c) a+a+((a+a+a+(a+a+a+a))

SDD to Produce AST or DAG

	PRODUCTION	SEMANTIC RULES
1)	$E \to E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2)	$E \to E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3)	$E \to T$	$E.node = T.node$
4)	$T \to (\ E\ )$	$T.node = E.node$
5)	$T \to \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6)	$T \to \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)$

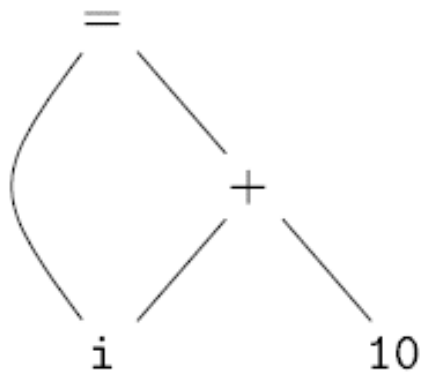
Figure 5.10: Constructing syntax trees for simple expressions

Figure 6.4: Syntax-directed definition to produce syntax trees or DAG's

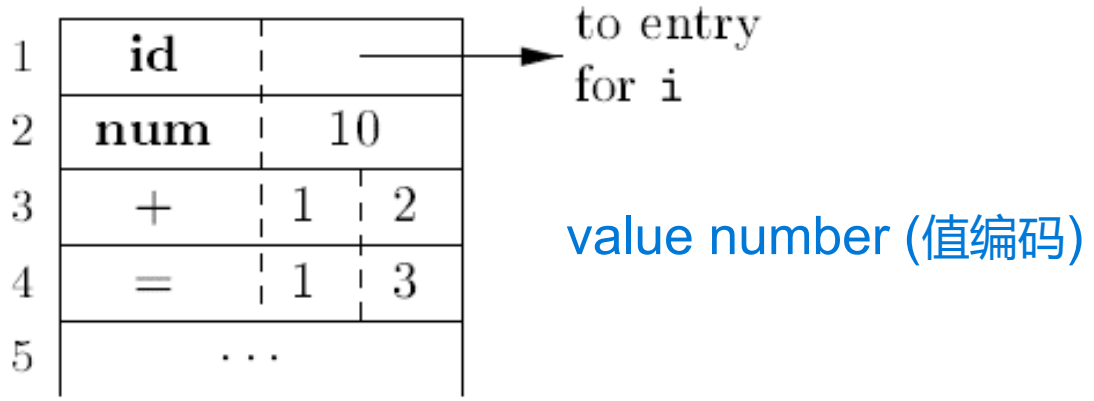
- 在构造新结点之前先检查该结点是否已存在。

6.1.2 The Value-Number Method for Constructing DAG's

- Often, the nodes of an AST or DAG are stored in an array of records.



(a) DAG



(b) Array.

Figure 6.6: Nodes of a DAG for $i = i + 10$ allocated in an array

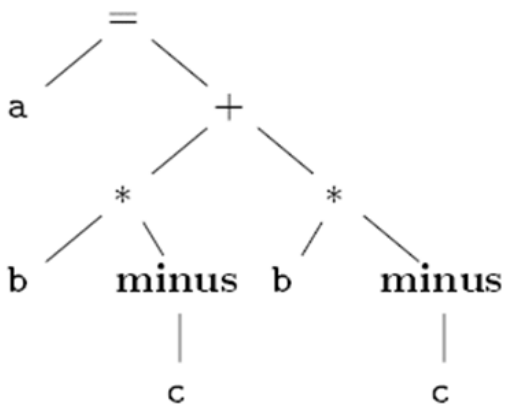
6.2 Three-Address Code

$$x + y * z \longrightarrow \begin{aligned} t_1 &= y * z \\ t_2 &= x + t_1 \end{aligned}$$

- Where t_1 and t_2 are compiler-generated temporary names.
- In three-address code, there is at most one operator on the right side of an instruction; that is, no built-up arithmetic expressions are permitted.

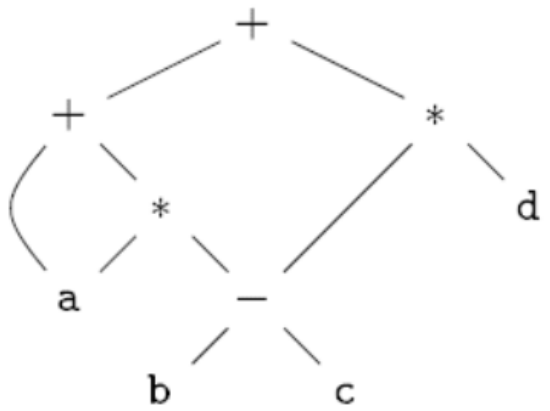
Example

- 三地址码是 AST 或 DAG 的线性表示形式。其中的（临时）名字对应于 AST 或 DAG 中的内部（运算符）节点。
- **Example** `a=b*-c+b*-c`



$$
\begin{aligned}
t_1 &= \text{minus } c \\
t_2 &= b * t_1 \\
t_3 &= \text{minus } c \\
t_4 &= b * t_3 \\
t_5 &= t_2 + t_4 \\
a &= t_5
\end{aligned}
$$

- **Example 6.4** `a+a*(b-c)+(b-c)*d`



$$
\begin{aligned}
t_1 &= b - c \\
t_2 &= a * t_1 \\
t_3 &= a + t_2 \\
t_4 &= t_1 * d \\
t_5 &= t_3 + t_4
\end{aligned}
$$

- **Example** `if (flag) x = -1 ; else x = 1;`
- **Example** `do i=i+1; while (a[i]<v);`

$$
\begin{aligned}
L: \quad & t_1 = i + 1 \\
& i = t_1 \\
& t_2 = i * 8 \\
& t_3 = a[\,t_2\,] \\
& \text{if } t_3 < v \text{ goto } L
\end{aligned}
$$

6.2.1 Addresses and Instructions

- 三地址码基于两个基本概念：地址和指令。

<table>
<tr><td>地址</td><td colspan="3">• 名字, 常量, 或编译器生成的临时名字</td><td rowspan="10">• 三地址码可以看作抽象的指令集，类似通用的RISC。
• 三地址码 / 三地址指令</td></tr>
<tr><td rowspan="9">指令</td><td colspan="3">• 常用的三地址指令</td></tr>
<tr><td>1</td><td>x = y op z</td><td rowspan="2">赋值指令</td></tr>
<tr><td>2</td><td>x = op y</td></tr>
<tr><td>3</td><td>x = y</td><td>复制指令</td></tr>
<tr><td>4</td><td>goto L</td><td>无条件转移指令</td></tr>
<tr><td>5</td><td>if x relop y goto L</td><td rowspan="2">条件转移指令</td></tr>
<tr><td>6</td><td>if x goto L / ifFalse x goto L</td></tr>
<tr><td>7</td><td>param x, call p,n, and return y</td><td>过程调用和返回</td></tr>
<tr><td>8</td><td>x=y[i] and x[i]=y</td><td>索引赋值</td></tr>
<tr><td>9</td><td>x=&y, x=*y and *x=y</td><td>地址和指针赋值</td></tr>
</table>

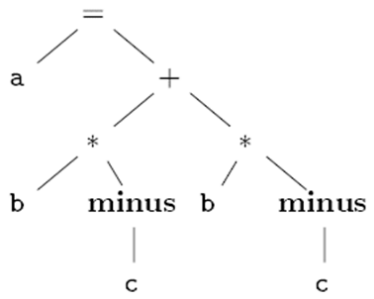
三地址码的实现

- 三地址码的实现方式：四元式、三元式和间接三元式。
  - 一个四元式有四个字段： op, arg_1, arg_2, 和 $result$ 。
  - 三元式使用指令的位置存储计算结果。
  - 间接三元式包含一个指向三元式指针的列表。
- 在优化编译器时，由于指令的位置常常会发生变化，因此四元式相对于三元式更有优势。
- 使用间接三元式，在优化编译器时如果指令的位置发生变化，三元式表不会变化。

三地址码	四元式
x = y op z	(op, y, z, x)
x = op z	(op, z, , x)
x = y	(=, y, , x)
goto L	(j, , , L)
if x relop y goto L	(jrelop, x, y, L)
if x goto L	(jNZ, x, , L)

6.2.2 四元式　　6.2.3 三元式

- **Example 6.6**　　a=b*-c+b*-c



$t_1 = \text{minus } c$
$t_2 = b * t_1$
$t_3 = \text{minus } c$
$t_4 = b * t_3$
$t_5 = t_2 + t_4$
$a = t_5$

	op	arg_1	arg_2	*result*	
0	minus	c		t_1	(minus, c, _, t_1)
1	*	b	t_1	t_2	(*, b, t_1, t_2)
2	minus	c		t_3	(minus, c, _, t_3)
3	*	b	t_3	t_4	(*, b, t_3, t_4)
4	+	t_2	t_4	t_5	(+, t_2, t_4, t_5)
5	=	t_5		a	(=, t_5, _, a)
		...			

	op	arg_1	arg_2	
0	minus	c		(minus, c, _)
1	*	b	(0)	(*, b, (0))
2	minus	c		(minus, c, _)
3	*	b	(2)	(*, b, (2))
4	+	(1)	(3)	(+, (1), (3))
5	=	a	(4)	(=, a, (4))
		...		

- **Exercise 6.2.1**　　Translate the arithmetic expression a + –(b + c) into:

  a) A syntax tree　　b) Quadruples　　c) Triples.

b) (+, b, c, t1)
(minus, c, _, t2)
(+, a, t2, t3)

c) t1 = b + c
t2 = minus t1
t3 = a + t2

- 带括号的数字表示指向相应三元式结构的指针

6.2.4 Static Single-Assignment Form

- 静态单赋值（SSA）是编译器的一种中间表示形式。静态单赋值可以很好的在未来被优化

```
y = 1        y1 = 1
y = 2        y2 = 2
x = y        x1 = y2
```

把对同一变量的多次赋值拆分成带版本号的新变量。

```
p = a + b          p1 = a + b
q = p - c          q1 = p1 - c
p = q * d          p2 = q1 * d
p = e - p          p3 = e - p2
q = p + q          q2 = p3 + q1
```

(a) Three-address code. (b) Static single-assignment form.

- 同一个变量可能在两个不同的控制流路径中被定义。

```
if (flag) x = -1 ; else x = 1;
y = x * a;
```

- SSA使用 ϕ函数来合并不同的控制流路径中的定义。

```
if (flag) x1 = -1 ; else x2 = 1;
x3 = ϕ(x1, x2);
y1 = x3 * a;
```

```
    if flag goto L1
    goto L2
L1: x1 = -1
    goto L3
L2: x2 = 1
L3: x3 = phi(x1, x2)
    y1 = x3 * a
```

6.2.4 SSA

- SSA要求每个变量仅被赋值一次，且每个变量在**使用**前都已被**定义**。
  - 定义 (define)：变量在一个表达式的左边，被赋值
  - 使用 (use) ：变量在一个表达式的右边，作为右值参与计算过程
- 优点
  - 简化变量的定义与使用之间的关系，让定义 - 使用（def-use）链唯一、清晰；
  - 简化数据流分析，有助于优化。
- 被 LLVM、gcc、Swift 等编译器广泛采用。

Grammar of Tiny

1. program → stmt-sequence
2. stmt-sequence → stmt-sequence**;** statement | statement
3. statement → if-stmt | repeat-stmt | assign-stmt | read-stmt | write-stmt
4. if-stmt → **if** exp **then** stmt-sequence **end** | **if** exp **then** stmt-sequence **else** stmt-sequence **end**
5. repeat-stmt → **repeat** stmt-sequence **until** exp
6. assign-stmt → **identifier :=** exp
7. read-stmt → **read identifier**
8. write-stmt → **write** exp
9. exp → simple-exp comparison-op simple-exp | simple-exp
10. comparison-op → **<** | **=**
11. simple-exp → simple-exp addop term | term
12. addop → **+** | **-**
13. term → term mulop factor | factor
14. mulop → ***** | **/**
15. factor → **(**exp**)** | **number** | **identifier**

```
x := 5;
if 0<x then
   fact := 1;
   repeat
      fact := fact * x;
      x := x - 1
   until x = 0;
end
```

Grammar of C-

1. program → declaration-list
2. declaration-list → declaration-list declaration | declaration
3. declaration → var-declaration | fun-declaration
4. var-declaration → type-specifier **ID;** | type-specifier **ID [NUM];**
5. type-specifier → **int** | **void**
6. fun-declaration → type-specifier **ID (**params**)** compound-stmt
7. params → param-list | **void**
8. param-list→ param-list , param | param
9. param → type-specifier **ID** | type-specifier **ID [ ]**
10. compound-stmt → **{** local-declarations statement-list **}**
11. local-declarations → local-declarations var-declaration | empty
12. statement-list → statement-list statement | empty
13. statement → expression-stmt | compound-stmt | selection-stmt | iteration-stmt | return-stmt
14. expression-stmt → expression **;** | **;**
15. selection-stmt → **if (**expression**)** statement | **if (**expression**)** statement **else** statement
16. iteration-stmt → **while (** expression **)** statement
17. return-stmt → **return;** | **return** expression**;**
18. expression → var = expression | simple-expression
19. var → **ID** | **ID [** expression **]**
20. simple-expression → additive-expression relop additive-expression | additive-expression
21. relop → **<=** | **<** | **>** | **>=** | **==** | **!=**
22. additive-expression → additive-expression addop term | term
23. addop → **+** | **-**
24. term → term mulop factor | factor
25. mulop → ***** | */*
26. factor → **(** expression **)** | var | call | **NUM**
27. call → **ID (** args **)**
28. args → arg-list | empty
29. arg-list → arg-list **,** expression | expression

Appendix A

program → *block*

block → { *decls stmts* }

decls → *decls decl* | **ε**

decl → *type* **id ;**

type → *type* [**num**] | **basic**

stmts → *stmts stmt* | **ε**

stmt → *loc* = *bool* **;**

| **if (** *bool* **)** *stmt*

| **if (** *bool* **)** *stmt* **else** *stmt*

| **while (** *bool* **)** *stmt*

| **do** *stmt* **while (** *bool* **) ;**

| **break ;**

| *block*

bool → *bool* || *join* | *join*

join → *join* && *equality* | *equality*

equality → *equality* == *rel* | *equality* != *rel* | *rel*

rel → *expr* < *expr* | *expr* <= *expr* | *expr* >= *expr* | *expr* > *expr* | *expr*

expr → *expr* + *term* | *expr* – *term* | *term*

term → *term* * *unary* | *term* / *unary* | *unary*

unary → **!** *unary* | – *unary* | *factor*

factor → **(** *bool* **)** | *loc* | **num** | **real** | **true** | **false**

```
1) {                    // File test
2)    int i; int j; float v; float x; float[100] a;
3)    while( true ) {
4)       do i = i+1; while( a[i] < v);
5)       do j = j-1; while( a[j] > v);
6)       if( i >= j ) break;
7)       x = a[i]; a[i] = a[j]; a[j] = x;
8)    }
9) }
```

```
 1) L1:L3:  i = i + 1
 2) L5:     t1 = i * 8
 3)         t2 = a [ t1 ]
 4)         if t2 < v goto L3
 5) L4:     j = j - 1
 6) L7:     t3 = j * 8
 7)         t4 = a [ t3 ]
 8)         if t4 > v goto L4
 9) L6:     iffalse i >= j goto L8
10) L9:    goto L2
11) L8:    t5 = i * 8
12)        x = a [ t5 ]
13) L10:   t6 = i * 8
14)        t7 = j * 8
15)        t8 = a [ t7 ]
16)        a [ t6 ] = t8
17) L11:   t9 = j * 8
18)        a [ t9 ] = x
19)        goto L1
20) L2:
```